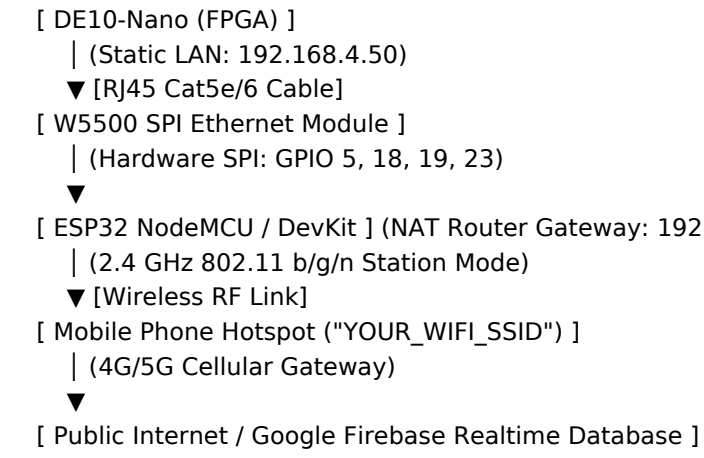
# ESP32-W5500 Router — public copy

Reformatted from the supplied router guide. Personal Wi-Fi settings have been replaced by placeholders.

Comprehensive Engineering Guide:
ESP32 + W5500 Ethernet NAT Router for
DE10-Nano Teleoperation

This engineering manual documents the complete deployment lifecycle of the ESP32-W5500 NAT router pipeline, designed to route secure outbound cloud telemetry (Google Firebase) from an Altera DE10-Nano (Cyclone V SoC-FPGA) through an upstream mobile hotspot.

1. System Architecture & Hardware Bill of Materials




Hardware Components

- Controller: ESP32 Development Board (30-pin or 38-pin DevKitC / NodeMCU-32S, Type-C or Micro-USB).
- Ethernet Physical Interface: Wiznet W5500 SPI Ethernet Breakout Module with integrated magnetics.
- Target Node: Terasic DE10-Nano SoC-FPGA (Angstrom Linux Kernel 4.1.33-ltsi).
- Power Supplies:
  - Independent 5V/2A DC Micro-USB / Type-C adapter dedicated to the ESP32 + W5500 assembly.
  - 5V/2A DC barrel jack power supply dedicated to the DE10-Nano.

2. SPI Pinout & Hardware Interconnects

Do not connect the W5500 to the ESP32 while flashing firmware over USB to avoid SPI bus contention. Wire the hardware only after flashing is complete.

Pinout Table

| W5500 Module Pin | ESP32 GPIO (30-Pin) | ESP32 GPIO (38-Pin) | Direction | Electrical Function |
|---|---|---|---|---|
| VCC / 3.3V | 3V3 | 3V3 (Pin 1) | Output $\rightarrow$ In | 3.3V Logic & Core Power |
| GND | GND | GND (Pin 14 / Pin 37) | — | Common System Ground |
| CS / NSS | GPIO 5 (D5) | GPIO 5 (IO5) | ESP32 $\rightarrow$ W5500 | SPI Slave Select / Chip Enable |
| SCK / SCLK | GPIO 18 (D18) | GPIO 18 (IO18) | ESP32 $\rightarrow$ | Serial Peripheral |

| | | | | |
|---|---|---|---|---|
| | | W5500 | Interface | Clock |
| MISO / SO | GPIO 19 (D19) | GPIO 19 (IO19) | W5500 $\rightarrow$ ESP32 | Master In / Slave Out (SPI Data) |
| MOSI / SI | GPIO 23 (D23) | GPIO 23 (IO23) | ESP32 $\rightarrow$ W5500 | Master Out / Slave In (SPI Data) |
| RST | NC (Leave Empty) | NC (Leave Empty) | — | Unconnected (Handled internally) |
| INT | NC (Leave Empty) | NC (Leave Empty) | — | Unconnected (Polling driver) |

3. Firmware Preparation & Acquisition

The router firmware utilizes the ESP-IDF Lightweight IP (lwIP) Network Address and Port Translation (NAPT) stack. Three separate binary partitions are required:

1. bootloader.bin: First-stage second-level bootloader initializing internal flash and memory maps.
2. partitions.bin: Partition table declaring flash layout, NVS space, and application partitions.
3. esp32_ethernet_router.bin: Main compiled application containing SPI W5500 driver, NAPT translation, DHCP server, and CLI parser.

4. Step-by-Step Flashing Procedure

Method A: Web Serial (Google Chrome / Microsoft Edge)

1. Disconnect all jumper wires between the ESP32 and the W5500 module.
2. Connect the ESP32 to the PC using a USB data cable.
3. Open a Chromium-based browser and navigate to: [https://espressif.github.io/esptool-js/](https://espressif.github.io/esptool-js/)
4. Configure connection parameters:
   ○ Baudrate: 115200 (or 921600 for high-speed writes).
5. Put ESP32 in ROM bootloader mode:
   ○ Press and hold the BOOT (or IO0) tactile button.
   ○ Press and release the EN (or RST) button once.
   ○ Release the BOOT button.
6. Click Connect in the web tool and select your serial port (e.g., COM13).
7. Click Erase Flash. Wait until the log displays:
   Plaintext
   Chip erase completed successfully.
8. In the Flash Program section, add three file rows with these exact memory offsets:

| Row | File | Offset (Hex) | Purpose |
|---|---|---|---|
| 1 | bootloader.bin | 0x1000 | Secondary bootloader |
| 2 | partitions.bin | 0x8000 | Partition scheme definition |
| 3 | esp32_ethernet_r | 0x20000 | Main |

Application
outer.bin binary

9. Critical Troubleshooting Rule: If the binary is flashed at 0x10000 when partitions.bin defines an OTA partition table, the ESP32 enters a rapid reset loop printing esp_image: image at 0x20000 has invalid magic byte (nothing flashed here?). The application binary must be flashed to 0x20000.
10. Click Program. Wait until all progress bars reach 100% and the log prints:
    Plaintext

    Leaving...
    Hard resetting via RTS pin...

11. Click Disconnect in the web interface.

5. Serial Configuration & Hotspot Uplink

1. Wire the W5500 module to the ESP32 according to the Section 2 pinout table.
2. Open Arduino IDE (or PuTTY):
   ○ Navigate to Tools $\rightarrow$ Port $\rightarrow$ select your port (e.g., COM13).
   ○ Open Tools $\rightarrow$ Serial Monitor (Ctrl + Shift + M).
   ○ Set the baud rate dropdown to 115200 baud.
   ○ Set the line-ending dropdown to Both NL & CR.
3. Press the EN / RST push button on the ESP32.
4. Verify the startup logs conclude with the command prompt:
   Plaintext
   esp32>

5. Ensure your phone hotspot is enabled with:
   ○ SSID: YOUR_WIFI_SSID
   ○ Band: 2.4 GHz only (turn on "Maximize Compatibility" if using iOS).
6. In the Serial Monitor input line, send the following sequence one by one:
   Plaintext
   set sta YOUR_WIFI_SSID YOUR_WIFI_PASSWORD
   save
   restart

7. Verify that the ESP32 connects to the hotspot upon rebooting:
   Plaintext
   I (...) esp_netif_handlers: sta ip: 192.168.x.x, mask: 255.255.255.0, gw: 192.168.x.x
   I (...) ESP32 Ethernet Router: got ip:192.168.x.x

8. Validate internet access directly from the ESP32 CLI:
   Plaintext
   ping 8.8.8.8

   Expected Result:
   Plaintext
   5 packets transmitted, 5 received, 0% packet loss

6. DE10-Nano Operating System Configuration

Connect the W5500 RJ45 port to the DE10-Nano RJ45 port using an Ethernet cable. Power the ESP32 via its dedicated 5V/2A supply. Connect to the DE10-Nano over PuTTY via its USB-UART

serial console (115200 baud, 8-N-1, root login).

6.1 Persistent Network Interface Configuration

Create the persistent systemd-networkd profile on the DE10-Nano so it automatically binds

to
the ESP32 router subnet on boot:

```
Bash
cat << 'EOF' > /etc/systemd/network/10-eth0.network
[Match]
Name=eth0

[Network]
Address=192.168.4.50/24
Gateway=192.168.4.1
DNS=8.8.8.8
DNS=1.1.1.1
EOF
```

Restart the network daemon to load the static configuration:

```
Bash
systemctl restart systemd-networkd
```

6.2 SSL/TLS Clock Rectification & /etc/rc.local Automation
The DE10-Nano lacks an onboard Real-Time Clock (RTC). If the board boots into its
2016/2017
kernel epoch, Google Firebase rejects all HTTPS/REST connections with
CertificateNotYetValidError.
To automate clock calibration and network verification at every boot, create an automated
/etc/rc.local script:

```
Bash
cat << 'EOF' > /etc/rc.local
#!/bin/sh -e


# 1. Base timestamp fallback (satisfies Firebase SSL certificates)
date -s "2026-09-28 09:00:00"

# 2. Re-assert persistent network stack
systemctl restart systemd-networkd

# 3. Synchronize exact atomic clock via NTP in background
(sleep 8 && ntpdate -u pool.ntp.org 2>/dev/null || rdate -s time.nist.gov) &

exit 0
EOF

chmod +x /etc/rc.local
```

7. Final End-to-End Diagnostic Sequence
To verify the full communication link at any time from the DE10-Nano terminal, execute the
following one-line test:

```
Bash
systemctl restart systemd-networkd && \
(ntpdate -u pool.ntp.org 2>/dev/null || date -s "2026-09-28 09:00:00") && \
ping -c 2 192.168.4.1 && ping -c 2 8.8.8.8
```

Expected Terminal Output

Plaintext

```
# 1. Gateway Ping Verification:
PING 192.168.4.1 (192.168.4.1) 56(84) bytes of data.
64 bytes from 192.168.4.1: icmp_seq=1 ttl=255 time=1.42 ms
64 bytes from 192.168.4.1: icmp_seq=2 ttl=255 time=1.18 ms

--- 192.168.4.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.182/1.303/1.424/0.121 ms


# 2. Public Internet Routing Verification:
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=58.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=61.2 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 58.412/59.806/61.201/1.394 ms
```

7.1 Cloud Database Verification
Verify HTTPS transactions directly against your Firebase endpoint:

Bash

```
curl -k -m 5
https://de10-iot-bridge-default-rtdb.asia-southeast1.firebasedatabase.app/teleop.json
```

If this command outputs the active joint coordinates JSON string without hanging, the communication pipeline is operating at 100% capacity.